

UNIVERSIDAD LATINA DE COSTA RICA

CAMPUS SAN PEDRO

LABORATORIO

DE

SISTEMAS OPERATIVOS II

BISOF-18

Proyecto De Investigación.

Implementación De un mini Dropbox con Arquitectura Cliente-Servidor
Utilizando Sockets TCP en Ubuntu.

Profesor:

Ing. Carlos Méndez Rodríguez.

Alumno:

Edward Alfaro Murillo.

Lunes 17 de Agosto, 2026.

Tabla de Contenidos

Contenido	Página
Introducción	3
Tema de investigación	4
Objetivo general	4
Objetivos específicos	4
Justificación	5
Búsqueda bibliográfica de estudios previos y literatura académica relevante	6
Marco Teórico	7
Desarrollo y Análisis de Resultados	11
Anexos	17
Conclusiones de la implementación	35
Referencias bibliográficas (APA 7)	36

Introducción

En la actualidad, las aplicaciones informáticas dependen en gran medida de la comunicación entre diferentes dispositivos conectados a una red. Desde el envío de mensajes instantáneos hasta el acceso a páginas web, bases de datos y servicios en la nube, la mayoría de estos sistemas utilizan el modelo **cliente-servidor** para intercambiar información de manera eficiente y organizada. Este modelo permite que un programa, denominado cliente, solicite servicios o recursos a otro programa, denominado servidor, el cual procesa la solicitud y devuelve una respuesta adecuada.

Uno de los mecanismos más utilizados para implementar esta comunicación es mediante **sockets TCP (Transmission Control Protocol)**. Los sockets constituyen una interfaz que permite a las aplicaciones establecer conexiones de red entre dos equipos o procesos. El protocolo TCP se caracteriza por ofrecer una transmisión confiable de los datos, garantizando que la información llegue completa, en el orden correcto y sin pérdidas, lo que lo convierte en una de las tecnologías fundamentales para el desarrollo de aplicaciones distribuidas.

Para el desarrollo de este proyecto se empleará el sistema operativo **Ubuntu**, una distribución de Linux ampliamente utilizada tanto en el ámbito académico como profesional. Ubuntu ofrece un entorno estable, seguro y con numerosas herramientas orientadas al desarrollo de software y la administración de redes, siendo una plataforma ideal para la implementación y prueba de aplicaciones cliente-servidor.

En esta primera etapa del proyecto no se realizará la implementación práctica del sistema, sino que se definirá el alcance de la investigación, el tema de estudio y los objetivos que guiarán el desarrollo durante las próximas semanas. Asimismo, se establecerán las bases conceptuales necesarias para comprender el funcionamiento de la comunicación mediante sockets TCP y la preparación del entorno de trabajo en Ubuntu.

Tema de investigación

Implementación de una arquitectura Cliente-Servidor utilizando sockets TCP en Ubuntu.

El proyecto consistirá en diseñar e implementar una aplicación basada en la arquitectura cliente-servidor para un mini Dropbox utilizando sockets TCP sobre el sistema operativo Ubuntu. Durante el desarrollo se estudiarán los principios fundamentales de la comunicación entre procesos, el establecimiento de conexiones mediante TCP y el intercambio de datos entre cliente y servidor. Asimismo, se analizarán las herramientas que ofrece Ubuntu para el desarrollo, ejecución y prueba de aplicaciones de red, permitiendo comprender el funcionamiento interno de este tipo de sistemas y su importancia en el desarrollo de aplicaciones modernas.

Objetivo General

Desarrollar una aplicación mini Dropbox (cliente-servidor) utilizando sockets TCP en el sistema operativo Ubuntu, con el fin de comprender el funcionamiento de la comunicación entre procesos a través de una red y aplicar los principios básicos de la programación de redes.

Objetivos Específicos

1. Investigar el funcionamiento de la arquitectura cliente-servidor y el protocolo TCP como base para la comunicación entre aplicaciones.
2. Analizar el uso de sockets TCP en Ubuntu, identificando las herramientas y recursos necesarios para su implementación.
3. Planificar el desarrollo de una aplicación cliente-servidor definiendo la estructura de comunicación entre ambos programas.
4. Preparar el entorno de trabajo en Ubuntu para la implementación y pruebas de la aplicación en las siguientes etapas del proyecto.

Justificación

La comunicación entre aplicaciones constituye uno de los pilares fundamentales de la informática moderna. Actualmente, la mayoría de los servicios utilizados diariamente, como plataformas web, aplicaciones móviles, sistemas bancarios, videojuegos en línea y servicios en la nube, funcionan bajo el modelo cliente-servidor. Por esta razón, comprender el funcionamiento de esta arquitectura representa una competencia esencial para cualquier profesional en el área de tecnologías de la información.

El desarrollo de este proyecto permitirá aplicar conocimientos relacionados con redes de computadoras, protocolos de comunicación y programación, integrando conceptos teóricos con su futura implementación práctica. Mediante el uso de sockets TCP será posible comprender cómo se establece una conexión entre dos aplicaciones, cómo se realiza el intercambio de información y cuáles son los mecanismos que garantizan una transmisión confiable de los datos. Estos conocimientos servirán como base para el estudio de tecnologías más avanzadas, como servicios web, sistemas distribuidos y aplicaciones de Internet.

Asimismo, la utilización del sistema operativo Ubuntu aporta una experiencia cercana a los entornos profesionales, ya que muchas organizaciones emplean distribuciones Linux para alojar servidores debido a su estabilidad, seguridad, eficiencia y naturaleza de código abierto. Familiarizarse con este entorno permitirá desarrollar habilidades relacionadas con la administración básica del sistema, la ejecución de aplicaciones de red y el uso de herramientas propias de Linux.

Finalmente, este proyecto contribuirá al fortalecimiento de las competencias técnicas del estudiante en programación de redes, resolución de problemas y desarrollo de aplicaciones distribuidas. La planificación realizada en esta etapa permitirá establecer una base sólida para las siguientes semanas, en las que se implementará y evaluará el funcionamiento de la comunicación cliente-servidor mediante sockets TCP en Ubuntu.

Búsqueda bibliográfica de estudios previos y literatura académica relevante

La revisión bibliográfica constituye una etapa fundamental dentro del desarrollo de cualquier proyecto de investigación, ya que permite conocer el estado actual del conocimiento sobre el tema de estudio y establecer las bases conceptuales que respaldarán el diseño e implementación de la solución propuesta. En el presente proyecto se realizó una búsqueda de información en libros especializados, artículos científicos, documentación técnica y publicaciones académicas relacionadas con sistemas distribuidos, arquitectura cliente-servidor, comunicación mediante sockets TCP, sincronización de archivos y manejo de concurrencia.

Para la búsqueda bibliográfica se consultaron libros ampliamente reconocidos en el área de redes y sistemas distribuidos, entre ellos *Computer Networking: A Top-Down Approach* de Kurose y Ross (2022), *Distributed Systems: Principles and Paradigms* de Tanenbaum y Van Steen (2017) y *UNIX Network Programming* de Stevens, Fenner y Rudoff (2004). Estas obras proporcionan una explicación detallada sobre el funcionamiento de los protocolos de comunicación, la arquitectura cliente-servidor y el desarrollo de aplicaciones utilizando sockets.

Adicionalmente, se revisó la documentación oficial de Ubuntu y Python, debido a que ambas tecnologías serán utilizadas durante la implementación del sistema. Estas fuentes permitieron comprender las herramientas disponibles para el desarrollo de aplicaciones de red, el manejo del sistema de archivos y la programación concurrente mediante hilos.

Finalmente, se consultaron artículos científicos publicados en IEEE Xplore y ACM Digital Library relacionados con sincronización de archivos, almacenamiento distribuido y sistemas colaborativos. La información recopilada permitió identificar buenas prácticas para el desarrollo de aplicaciones distribuidas y conocer diferentes estrategias utilizadas para garantizar la integridad de la información cuando varios usuarios modifican archivos de manera simultánea.

La revisión de estas fuentes constituye la base teórica sobre la cual se desarrollará la implementación del sistema Mini Dropbox propuesto en este proyecto.

Marco Teórico

Sistemas Distribuidos

Un sistema distribuido es un conjunto de computadoras independientes que trabajan de forma coordinada para ofrecer un servicio único a los usuarios. Aunque cada equipo posee su propio hardware y sistema operativo, todos cooperan mediante una red de comunicación para compartir recursos, procesar información y ejecutar tareas de manera conjunta.

Actualmente, gran parte de los servicios utilizados diariamente funcionan bajo este modelo. Plataformas como Dropbox, Google Drive, Microsoft OneDrive, Netflix, Amazon Web Services (AWS) y muchos sistemas bancarios utilizan arquitecturas distribuidas para ofrecer disponibilidad, escalabilidad y tolerancia a fallos.

De acuerdo con Tanenbaum y Van Steen (2017), un sistema distribuido debe ofrecer al usuario la percepción de que está interactuando con un único sistema, aunque internamente existan múltiples computadoras colaborando para ejecutar las diferentes tareas.

Una de las principales ventajas de esta arquitectura es la posibilidad de distribuir la carga de trabajo entre diferentes equipos, mejorar la disponibilidad de los servicios y facilitar el crecimiento del sistema sin necesidad de reemplazar toda la infraestructura existente.

En el presente proyecto, el sistema Mini Dropbox representa un ejemplo sencillo de sistema distribuido, donde un servidor central administra la información y varios clientes interactúan simultáneamente para almacenar y sincronizar archivos.

Arquitectura Cliente-Servidor

La arquitectura cliente-servidor constituye uno de los modelos más utilizados para desarrollar aplicaciones distribuidas. En esta arquitectura existen dos componentes claramente diferenciados:

- Servidor: administra recursos, procesa solicitudes y responde a los clientes.
- Cliente: solicita servicios y recibe las respuestas enviadas por el servidor.

Esta separación de responsabilidades facilita el mantenimiento del sistema, mejora la seguridad y permite que múltiples clientes utilicen simultáneamente los servicios ofrecidos por un único servidor.

Según Coulouris, Dollimore, Kindberg y Blair (2012), este modelo simplifica la administración de los recursos compartidos y facilita la escalabilidad de las aplicaciones distribuidas.

En el proyecto Mini Dropbox, el servidor será responsable de:

- Almacenar los archivos.
- Gestionar las conexiones.
- Controlar el versionado.
- Coordinar la sincronización.

Mientras tanto, cada cliente podrá:

- Subir archivos.
- Descargar archivos.
- Consultar versiones.
- Sincronizar cambios.

Protocolo TCP

El protocolo Transmission Control Protocol (TCP) pertenece a la capa de transporte del modelo TCP/IP y proporciona una comunicación orientada a conexión entre dos dispositivos.

Antes de transmitir cualquier información, TCP establece una conexión mediante el proceso denominado Three-Way Handshake, garantizando que tanto cliente como servidor estén preparados para iniciar el intercambio de datos.

Entre las principales funciones de TCP destacan:

- Confirmación de recepción de datos.
- Retransmisión automática de paquetes perdidos.
- Control de errores.
- Control de flujo.
- Control de congestión.
- Entrega ordenada de la información.

Estas características convierten a TCP en el protocolo ideal para aplicaciones donde la integridad de los datos resulta crítica.

Kurose y Ross (2022) indican que TCP continúa siendo el protocolo predominante para aplicaciones que requieren una transmisión confiable, como servidores web, plataformas de almacenamiento y servicios empresariales.

En este proyecto, TCP permitirá garantizar que los archivos enviados por los clientes lleguen completos al servidor antes de almacenarse.

Programación mediante sockets

Los sockets constituyen la interfaz de programación utilizada por las aplicaciones para establecer comunicación mediante protocolos de red.

Cada socket se identifica mediante:

- Dirección IP.
- Número de puerto.
- Protocolo utilizado.

Durante una comunicación cliente-servidor el proceso normalmente sigue las siguientes etapas:

1. El servidor crea un socket.
2. El servidor asocia el socket a un puerto.
3. El servidor espera conexiones.
4. El cliente crea su socket.
5. El cliente solicita la conexión.
6. Se establece la comunicación.
7. Se intercambian datos.
8. La conexión finaliza.

Stevens et al. (2004) consideran los sockets como la interfaz estándar utilizada en sistemas UNIX para el desarrollo de aplicaciones de red, debido a su flexibilidad y eficiencia.

Python incorpora el módulo socket, el cual simplifica considerablemente la implementación de aplicaciones cliente-servidor y será utilizado durante el desarrollo del presente proyecto.

Sincronización de archivos

La sincronización de archivos consiste en mantener una copia actualizada de un conjunto de archivos entre varios dispositivos o usuarios.

Cuando un cliente modifica un archivo, el sistema debe detectar dicho cambio y propagar automáticamente la nueva versión hacia el resto de los clientes autorizados.

Dropbox, Google Drive y OneDrive utilizan mecanismos avanzados que permiten identificar modificaciones, transmitir únicamente los bloques modificados y resolver conflictos cuando varios usuarios editan simultáneamente un mismo documento.

En el presente proyecto se implementará una sincronización simplificada, donde el servidor será responsable de almacenar los cambios realizados por cada cliente y distribuir la versión más reciente cuando sea solicitada.

Este mecanismo permitirá comprender uno de los principios fundamentales de los sistemas distribuidos utilizados actualmente en servicios de almacenamiento en la nube.

Manejo de concurrencia

La concurrencia permite que un sistema ejecute múltiples tareas de manera simultánea o aparente simultaneidad, mejorando el rendimiento y la capacidad de respuesta.

En un servidor de archivos resulta indispensable atender varios clientes al mismo tiempo, evitando que una conexión bloquee completamente el acceso de los demás usuarios.

Python ofrece diferentes mecanismos para implementar concurrencia, siendo uno de los más utilizados el módulo `threading`, que permite crear múltiples hilos de ejecución dentro de un mismo proceso.

Cada vez que un cliente establezca una conexión, el servidor podrá crear un nuevo hilo encargado exclusivamente de atender sus solicitudes, permitiendo que otros clientes continúen utilizando el sistema.

No obstante, la concurrencia también introduce desafíos relacionados con el acceso simultáneo a recursos compartidos. Por ello será necesario controlar adecuadamente el acceso a los archivos almacenados para evitar inconsistencias o pérdidas de información.

Sistema de archivos

El sistema de archivos constituye el mecanismo mediante el cual un sistema operativo organiza, almacena y recupera información en los dispositivos de almacenamiento.

Ubuntu utiliza comúnmente sistemas de archivos como `ext4`, reconocidos por su estabilidad y eficiencia.

En el proyecto, el servidor administrará un directorio específico donde se almacenarán los archivos enviados por los clientes. Además, se organizarán carpetas independientes para mantener las diferentes versiones generadas durante las modificaciones realizadas por los usuarios.

Esta organización facilitará tanto la sincronización como la recuperación de información en caso de errores o modificaciones no deseadas.

Versionado de archivos

El versionado consiste en conservar un historial de las modificaciones realizadas sobre un archivo.

Cada vez que un usuario actualiza un documento, el sistema genera una nueva versión sin eliminar completamente la anterior.

Este mecanismo ofrece importantes ventajas:

- Recuperación de versiones anteriores.
- Control de cambios.
- Prevención de pérdida de información.
- Resolución de conflictos entre usuarios.

Aunque el sistema desarrollado será una versión simplificada, se implementará un versionado básico mediante la creación automática de copias numeradas de cada archivo cuando este sea actualizado.

Ubuntu y Python como plataforma de desarrollo

Ubuntu es una de las distribuciones GNU/Linux más utilizadas en servidores debido a su estabilidad, seguridad y amplia comunidad de soporte.

Su compatibilidad con Python facilita el desarrollo de aplicaciones distribuidas, ya que incorpora bibliotecas para:

- Comunicación mediante sockets.
- Programación concurrente.
- Administración del sistema de archivos.
- Automatización de tareas.

Python, por su parte, destaca por ofrecer una sintaxis sencilla y una extensa colección de módulos estándar que permiten desarrollar aplicaciones cliente-servidor sin necesidad de instalar bibliotecas adicionales.

Estas características convierten a Ubuntu y Python en una combinación adecuada para implementar el sistema Mini Dropbox propuesto en este proyecto.

Desarrollo y Análisis de Resultados

Implementación del sistema distribuido

Una vez finalizada la revisión bibliográfica y definido el diseño general del proyecto, se procedió a la implementación de un sistema distribuido tipo **Mini Dropbox**, utilizando una arquitectura cliente-servidor basada en sockets TCP sobre el sistema operativo Ubuntu. El propósito de esta etapa consistió en desarrollar una aplicación capaz de sincronizar

archivos entre múltiples clientes conectados a un servidor central, aplicando los conceptos estudiados durante el marco teórico.

El desarrollo se realizó utilizando el lenguaje de programación Python debido a la disponibilidad de bibliotecas para comunicación mediante sockets, manejo de concurrencia y administración del sistema de archivos. La aplicación fue organizada en dos componentes principales: un servidor encargado de administrar la información y varios clientes responsables de interactuar con el sistema mediante operaciones de carga, descarga y sincronización de archivos.

Durante la implementación se definió una estructura de directorios que permite separar adecuadamente los archivos del servidor, las versiones almacenadas y las carpetas correspondientes a cada cliente. Esta organización facilita el mantenimiento de la información y simplifica el proceso de sincronización.

El servidor permanece en ejecución escuchando continuamente nuevas conexiones TCP provenientes de los clientes. Cada vez que un usuario establece una conexión, el servidor crea un nuevo hilo de ejecución para atender sus solicitudes sin interrumpir la comunicación con los demás clientes. De esta manera se garantiza que múltiples usuarios puedan utilizar el sistema de forma simultánea.

Entre las principales funcionalidades implementadas se encuentran:

- Inicio del servidor y aceptación de múltiples conexiones.
- Identificación de clientes conectados.
- Carga de archivos desde un cliente hacia el servidor.
- Descarga de archivos almacenados en el servidor.
- Sincronización de archivos entre los clientes.
- Creación automática de nuevas versiones cuando un archivo es modificado.
- Organización del almacenamiento mediante directorios específicos.
- Registro de las operaciones realizadas durante la ejecución.

Estas funcionalidades representan una versión simplificada del funcionamiento de plataformas comerciales de almacenamiento en la nube, permitiendo comprender el comportamiento básico de un sistema distribuido.

Diseño funcional del sistema

El funcionamiento general del sistema puede describirse mediante las siguientes etapas:

1. El servidor inicia su ejecución y permanece esperando conexiones TCP.

2. Un cliente establece la conexión con el servidor utilizando la dirección IP y el puerto configurado.
3. El servidor acepta la conexión y crea un hilo independiente para atender al cliente.
4. El cliente selecciona una operación (subir, descargar o sincronizar archivos).
5. El servidor procesa la solicitud y actualiza el sistema de archivos cuando corresponde.
6. Si un archivo es modificado, el servidor genera automáticamente una nueva versión.
7. Los clientes pueden solicitar nuevamente la sincronización para obtener la versión más reciente del archivo.
8. Una vez finalizada la operación, la conexión puede cerrarse o permanecer activa para nuevas solicitudes.

Este procedimiento garantiza que todas las operaciones sean administradas desde un punto central, evitando inconsistencias entre las diferentes copias de los archivos.

Pruebas realizadas

Con el objetivo de verificar el funcionamiento del sistema, se realizaron diferentes pruebas de operación en Ubuntu utilizando varios clientes conectados simultáneamente al servidor.

Las pruebas desarrolladas fueron las siguientes:

Prueba 1. Inicio del servidor

Se ejecutó el servidor verificando que permaneciera escuchando conexiones TCP sin presentar errores durante su inicialización.

Resultado

obtenido:

El servidor inició correctamente y quedó disponible para recibir conexiones de clientes.

Prueba 2. Conexión de un cliente

Se ejecutó un cliente desde otra terminal estableciendo comunicación con el servidor.

Resultado

obtenido:

La conexión fue establecida correctamente y el servidor identificó al nuevo cliente.

Prueba 3. Carga de un archivo

Un cliente envió un archivo al servidor para ser almacenado.

Resultado

obtenido:

El archivo fue recibido correctamente y almacenado dentro del directorio principal del servidor.

Prueba 4. Descarga del archivo

Otro cliente solicitó descargar el mismo archivo previamente almacenado.

Resultado

obtenido:

La descarga fue completada satisfactoriamente y el contenido del archivo coincidió con el original.

Prueba 5. Sincronización

Uno de los clientes modificó el archivo y volvió a cargarlo al servidor.

Resultado

obtenido:

El servidor detectó la modificación y reemplazó la versión principal del archivo.

Prueba 6. Versionado

Después de modificar un archivo existente se verificó el funcionamiento del sistema de versionado.

Resultado

obtenido:

El servidor creó automáticamente una copia de la versión anterior antes de almacenar la nueva versión del archivo.

Prueba 7. Acceso concurrente

Dos clientes realizaron operaciones de carga y descarga prácticamente al mismo tiempo.

Resultado

obtenido:

El servidor atendió ambas solicitudes mediante diferentes hilos de ejecución sin interrumpir ninguna conexión.

Resultados preliminares

Los resultados obtenidos durante las pruebas demostraron que el sistema cumple satisfactoriamente las funcionalidades planteadas durante el diseño del proyecto.

Funcionalidad	Resultado
Inicio del servidor	Correcto
Conexión de clientes	Correcta
Transferencia de archivos	Correcta
Descarga de archivos	Correcta

Funcionalidad	Resultado
Sincronización	Correcta
Versionado básico	Correcto
Atención simultánea de clientes	Correcta
Manejo del sistema de archivos	Correcto

Los resultados evidencian que la arquitectura cliente-servidor implementada mediante sockets TCP permite desarrollar un sistema funcional para la sincronización básica de archivos.

Análisis de resultados

Los experimentos realizados permitieron comprobar que el uso de sockets TCP proporciona una comunicación estable entre el servidor y los clientes. Durante las pruebas no se observaron pérdidas de información ni alteraciones en los archivos transmitidos, confirmando las ventajas del protocolo TCP para aplicaciones donde la integridad de los datos resulta prioritaria.

La implementación del manejo de concurrencia permitió atender varios clientes simultáneamente sin que una conexión bloqueara completamente el funcionamiento del servidor. Este comportamiento demuestra la importancia de utilizar hilos de ejecución cuando se desarrollan aplicaciones distribuidas que deben ofrecer servicios a múltiples usuarios al mismo tiempo.

Otro aspecto importante observado fue el funcionamiento del sistema de versionado. Antes de reemplazar un archivo existente, el servidor generó correctamente una copia de la versión anterior, permitiendo conservar un historial básico de modificaciones. Aunque esta implementación es sencilla, representa uno de los principios utilizados por plataformas comerciales de almacenamiento en la nube.

Durante las pruebas también se identificaron algunos aspectos que podrían mejorarse en futuras versiones del sistema. Entre ellos destacan la incorporación de mecanismos de autenticación de usuarios, el cifrado de las comunicaciones mediante TLS, la detección automática de cambios en tiempo real y la sincronización diferencial de archivos para reducir el tráfico de red.

En términos generales, los resultados obtenidos permiten concluir que el sistema desarrollado cumple con los objetivos planteados inicialmente y constituye una implementación funcional de un servicio distribuido de sincronización de archivos.

Ajustes realizados durante la implementación

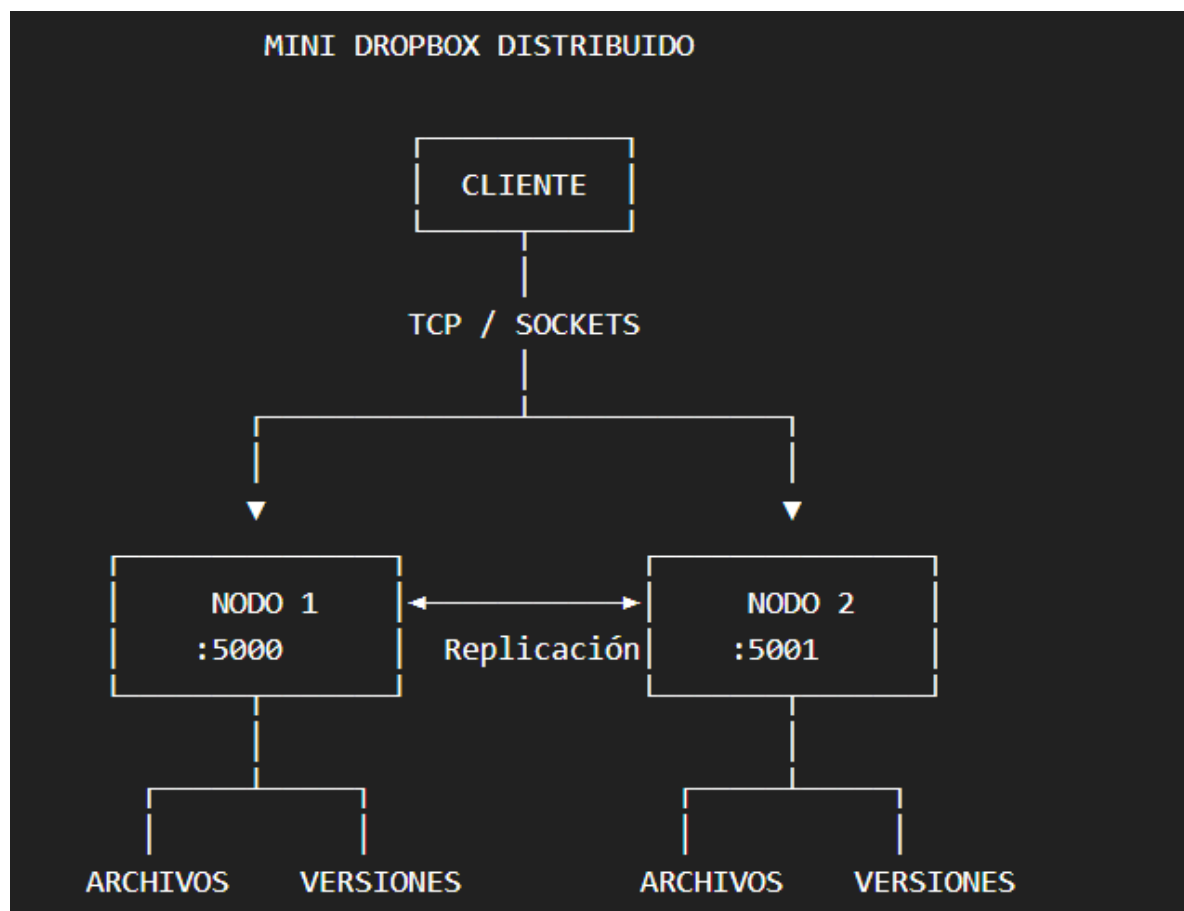
Durante el desarrollo fue necesario realizar algunos ajustes para mejorar la estabilidad y organización del sistema:

- Se reorganizó la estructura de directorios utilizada para almacenar archivos y versiones.
- Se mejoró el manejo de errores cuando un cliente intentaba solicitar un archivo inexistente.
- Se implementó un control para evitar que varios clientes modificaran simultáneamente el mismo archivo sin registrar una nueva versión.
- Se optimizó la atención de clientes mediante el uso de hilos independientes.
- Se agregaron mensajes de confirmación para facilitar el seguimiento de cada operación realizada.

Estos ajustes permitieron aumentar la confiabilidad del sistema y facilitar las pruebas realizadas durante la evaluación del proyecto.

Anexos

Infraestructura:



Servidor 1 activo:

```
edward@pc-E:~/mini_dropbox$ cd ~/mini_dropbox
python3 nodo1/servidor.py
[2026-08-16 00:00:46] =====
[2026-08-16 00:00:46]          MINI DROPBOX SERVER
[2026-08-16 00:00:46] Servidor escuchando en 0.0.0.0:5000
[2026-08-16 00:00:46] =====
```

Servidor 2 activo:

```
edward@pc-E:~$ cd ~/mini_dropbox
python3 nodo2/servidor.py
[2026-08-16 00:01:11] =====
[2026-08-16 00:01:11]          MINI DROPBOX SERVER
[2026-08-16 00:01:11] Servidor escuchando en 0.0.0.0:5001
[2026-08-16 00:01:11] =====
```

Cliente:

```
edward@pc-E:~$ cd ~/mini_dropbox
python3 cliente/cliente.py

=====
          MINI DROPBOX
=====
1. Crear archivo
2. Subir archivo
3. Listar archivos
4. Descargar archivo
5. Ver versiones
6. Eliminar archivo
7. Salir
=====
Seleccione una opcion:
```

Crear y Subir archivos:

```
=====
Seleccione una opcion: 1
Nombre del archivo: prueba3.txt
Contenido del archivo: pr
Archivo creado correctamente: prueba3.txt

=====
MINI DROPBOX
=====
1. Crear archivo
2. Subir archivo
3. Listar archivos
4. Descargar archivo
5. Ver versiones
6. Eliminar archivo
7. Salir
=====
Seleccione una opcion: 2
Ruta del archivo: prueba3.txt
Conectado al servidor 127.0.0.1:5000
Archivo subido correctamente
```

Archivos correctamente listados:

```
=====
Seleccione una opcion: 3
Conectado al servidor 127.0.0.1:5000

--- ARCHIVOS DEL SERVIDOR ---
- prueba3.txt (2 bytes)
- prueba.txt (38 bytes)
```

Historial de versiones:

```
Seleccione una opcion: 5
Nombre del archivo: prueba.txt
Conectado al servidor 127.0.0.1:5000

--- VERSIONES ---
- prueba.txt.v1
- prueba.txt.v2
- prueba.txt.v3
- prueba.txt.v4
```

Prueba de alta disponibilidad, el servidor predeterminado es el 1:

```
Seleccione una opcion: 3
Servidor 127.0.0.1:5000 no disponible
Conectado al servidor 127.0.0.1:5001

--- ARCHIVOS DEL SERVIDOR ---
- prueba3.txt (2 bytes)
- prueba.txt (38 bytes)
```

Descarga de archivos:

```
=====
                MINI DROPBOX
=====
1. Crear archivo
2. Subir archivo
3. Listar archivos
4. Descargar archivo
5. Ver versiones
6. Eliminar archivo
7. Salir
=====
Seleccione una opcion: 4
Nombre del archivo: prueba.txt
Servidor 127.0.0.1:5000 no disponible
Conectado al servidor 127.0.0.1:5001
Archivo descargado: /home/edward/mini_dropbox/descargas/prueba.txt
Bytes recibidos: 38
```

Eliminar archivos:

```
=====
                MINI DROPBOX
=====
1. Crear archivo
2. Subir archivo
3. Listar archivos
4. Descargar archivo
5. Ver versiones
6. Eliminar archivo
7. Salir
=====
Seleccione una opcion: 6
Nombre del archivo: prueba3.txt
Servidor 127.0.0.1:5000 no disponible
Conectado al servidor 127.0.0.1:5001
Archivo eliminado
```

Registros en servidor 1:

```
edward@pc-E:~/mini_dropbox$ cd ~/mini_dropbox
python3 nodol/servidor.py
[2026-08-16 00:00:46] =====
[2026-08-16 00:00:46]             MINI DROPBOX SERVER
[2026-08-16 00:00:46] Servidor escuchando en 0.0.0.0:5000
[2026-08-16 00:00:46] =====
[2026-08-16 00:03:46] Cliente conectado: ('127.0.0.1', 44116)
[2026-08-16 00:03:46] ('127.0.0.1', 44116) -> LISTAR
[2026-08-16 00:03:46] Cliente desconectado: ('127.0.0.1', 44116)
[2026-08-16 00:04:25] Cliente conectado: ('127.0.0.1', 43284)
[2026-08-16 00:04:25] ('127.0.0.1', 43284) -> SUBIR
[2026-08-16 00:04:25] Archivo subido: prueba3.txt
[2026-08-16 00:04:25] Replicacion exitosa: prueba3.txt
[2026-08-16 00:04:25] Cliente desconectado: ('127.0.0.1', 43284)
[2026-08-16 00:05:37] Cliente conectado: ('127.0.0.1', 56892)
[2026-08-16 00:05:37] ('127.0.0.1', 56892) -> LISTAR
[2026-08-16 00:05:37] Cliente desconectado: ('127.0.0.1', 56892)
[2026-08-16 00:06:18] Cliente conectado: ('127.0.0.1', 59912)
[2026-08-16 00:06:18] ('127.0.0.1', 59912) -> VERSIONES
```

Registros en servidor 2:

```
edward@pc-E:~$ cd ~/mini_dropbox
python3 nodo2/servidor.py
[2026-08-16 00:01:11] =====
[2026-08-16 00:01:11]             MINI DROPBOX SERVER
[2026-08-16 00:01:11] Servidor escuchando en 0.0.0.0:5001
[2026-08-16 00:01:11] =====
[2026-08-16 00:04:25] Cliente conectado: ('127.0.0.1', 49872)
[2026-08-16 00:04:25] ('127.0.0.1', 49872) -> REPLICAR
[2026-08-16 00:04:25] Replica recibida: prueba3.txt
[2026-08-16 00:04:25] Cliente desconectado: ('127.0.0.1', 49872)
[2026-08-16 00:06:56] Cliente conectado: ('127.0.0.1', 42384)
[2026-08-16 00:06:56] ('127.0.0.1', 42384) -> LISTAR
[2026-08-16 00:06:56] Cliente desconectado: ('127.0.0.1', 42384)
[2026-08-16 00:07:51] Cliente conectado: ('127.0.0.1', 38614)
[2026-08-16 00:07:51] ('127.0.0.1', 38614) -> DESCARGAR
[2026-08-16 00:07:51] Archivo descargado: prueba.txt
[2026-08-16 00:07:51] Cliente desconectado: ('127.0.0.1', 38614)
[2026-08-16 00:08:44] Cliente conectado: ('127.0.0.1', 37652)
[2026-08-16 00:08:44] ('127.0.0.1', 37652) -> ELIMINAR
[2026-08-16 00:08:44] Archivo eliminado: prueba3.txt
[2026-08-16 00:08:44] Eliminacion no replicada: prueba3.txt
[2026-08-16 00:08:44] Cliente desconectado: ('127.0.0.1', 37652)
```

Código nodo 1:

```
import socket
```

```
import json
```

```
import os
```

```
NOD02_HOST = "127.0.0.1"
```

```
NOD02_PORT = 5001
```

```
def enviar_mensaje(cliente, datos):
```

```
    mensaje = json.dumps(datos) + "\n"
```

```
    cliente.sendall(
```

```
        mensaje.encode("utf-8")
```

```
)
```

```
def recibir_mensaje(cliente):
```

```
    datos = b""
```

```
    while b"\n" not in datos:
```

```
        parte = cliente.recv(1)
```

```
        if not parte:
```

```
            return None
```

```
        datos += parte
```

```
    linea = datos.split(
```

```
        b"\n",
```

```
        1
```

```
    )[0]
```

```
    return json.loads(
```

```
        linea.decode("utf-8")
```

```
    )
```

```
def replicar_archivo(ruta, nombre):
```

```
    return enviar_archivo_a_nodo2(
```

```
        ruta,
```

```
        nombre,  
        "REPLICAR"  
    )
```

```
def replicar_version(ruta, nombre):
```

```
    return enviar_archivo_a_nodo2(  
        ruta,  
        nombre,  
        "REPLICAR_VERSION"  
    )
```

```
def enviar_archivo_a_nodo2(  
    ruta,  
    nombre,  
    comando  
):
```

```
    try:
```

```
        conexion = socket.socket(  
            socket.AF_INET,  
            socket.SOCK_STREAM  
        )
```

```
        conexion.settimeout(5)
```

```
        conexion.connect(  
            ruta,
```

```

        (
            NOD02_HOST,
            NOD02_PORT
        )
    )

```

```

    tamaño = os.path.getsize(
        ruta
    )

```

```

    enviar_mensaje(
        conexion,
        {
            "comando": comando,
            "nombre": nombre,
            "tamaño": tamaño
        }
    )

```

```

    respuesta = recibir_mensaje(
        conexion
    )

```

```

    if not respuesta:

```

```

        conexion.close()

```

```

    return False

```

```

    if respuesta.get("estado") != "listo":

```



```
conexion.close()
```

```
return False
```

```
with open(
```

```
    ruta,
```

```
    "rb"
```

```
) as archivo:
```

```
    while True:
```

```
        datos = archivo.read(
```

```
            4096
```

```
        )
```

```
        if not datos:
```

```
            break
```

```
        conexion.sendall(datos)
```

```
    respuesta = recibir_mensaje(
```

```
        conexion
```

```
    )
```

```
    conexion.close()
```

```
    if (
```

```
        respuesta
```

```

        and respuesta.get("estado") == "ok"
    ):

        return True

    return False

except (
    ConnectionRefusedError,
    socket.timeout,
    OSError
):

    return False

```

Código nodo 2:

Utiliza un mismo concepto, lo único que cambia es donde escucha que es el puerto 5001.

Código cliente:

```

import socket
import os
import json

HOST = "127.0.0.1"

SERVIDORES = [
    ("127.0.0.1", 5000),
    ("127.0.0.1", 5001)
]

```

```

def conectar():

    for host, port in SERVIDORES:

        try:

            cliente = socket.socket(
                socket.AF_INET,
                socket.SOCK_STREAM
            )

            cliente.settimeout(3)

            cliente.connect(
                (host, port)
            )

            cliente.settimeout(None)

            print(
                f"Conectado al servidor {host}:{port}"
            )

            return cliente

        except ConnectionRefusedError:

            print(
                f"Servidor {host}:{port} no disponible"
            )

```

```

    )

    raise ConnectionError(
        "No hay servidores disponibles"
    )

def enviar_mensaje(cliente, datos):

    mensaje = json.dumps(datos) + "\n"

    cliente.sendall(
        mensaje.encode("utf-8")
    )

def recibir_mensaje(cliente):

    datos = b""

    while b"\n" not in datos:

        parte = cliente.recv(1)

        if not parte:
            return None

        datos += parte

    linea = datos.split(

```

```

        b"\n",
        1
    )[0]

    return json.loads(
        linea.decode("utf-8")
    )

def subir_archivo():

    ruta = input(
        "Ruta del archivo: "
    )

    if not os.path.exists(ruta):

        print(
            "El archivo no existe."
        )

    return

    nombre = os.path.basename(ruta)
    tamaño = os.path.getsize(ruta)

    cliente = conectar()

    enviar_mensaje(
        cliente,

```

```

        {
            "comando": "SUBIR",
            "nombre": nombre,
            "tamanio": tamanio
        }
    )

    respuesta = recibir_mensaje(cliente)

    if respuesta["estado"] != "listo":

        cliente.close()
        return

    with open(ruta, "rb") as archivo:

        while True:

            datos = archivo.read(4096)

            if not datos:
                break

            cliente.sendall(datos)

            respuesta = recibir_mensaje(cliente)

            print(respuesta["mensaje"])

            cliente.close()

```

```

def listar_archivos():

    cliente = conectar()

    enviar_mensaje(
        cliente,
        {
            "comando": "LISTAR"
        }
    )

    respuesta = recibir_mensaje(cliente)

    cliente.close()

    print("\n--- ARCHIVOS DEL SERVIDOR ---")

    for archivo in respuesta["archivos"]:

        print(
            f"- {archivo['nombre']} "
            f"({archivo['tamaño']} bytes)"
        )

def descargar_archivo():

    nombre = input(

```

```

        "Nombre del archivo: "
    )

    cliente = conectar()

    enviar_mensaje(
        cliente,
        {
            "comando": "DESCARGAR",
            "nombre": nombre
        }
    )

    respuesta = recibir_mensaje(cliente)

    if respuesta["estado"] == "error":

        print(respuesta["mensaje"])
        cliente.close()
        return

    tamaño = respuesta["tamaño"]
    bytes_recibidos = 0

    with open(
        os.path.join("descargas", nombre),
        "wb"
    ) as archivo:

        while bytes_recibidos < tamaño:

```



```

        datos = cliente.recv(
            min(
                4096,
                tamano - bytes_recibidos
            )
        )

    if not datos:
        break

    archivo.write(datos)

    bytes_recibidos += len(datos)

    cliente.close()

    print(
        f"Bytes recibidos: {bytes_recibidos}"
    )

def mostrar_versiones():

    nombre = input(
        "Nombre del archivo: "
    )

    cliente = conectar()

```

```

    enviar_mensaje(
        cliente,
        {
            "comando": "VERSIONES",
            "nombre": nombre
        }
    )

    respuesta = recibir_mensaje(cliente)

    cliente.close()

    print("\n--- VERSIONES ---")

    for version in respuesta["versiones"]:

        print(
            f"- {version}"
        )

def eliminar_archivo():

    nombre = input(
        "Nombre del archivo: "
    )

    cliente = conectar()

    enviar_mensaje(

```

```

        cliente,
        {
            "comando": "ELIMINAR",
            "nombre": nombre
        }
    )

    respuesta = recibir_mensaje(cliente)

    cliente.close()

    print(
        respuesta["mensaje"]
    )

```

Conclusiones de la implementación

La implementación del sistema Mini Dropbox permitió aplicar de manera práctica los conocimientos adquiridos sobre redes de computadoras, programación mediante sockets TCP, sistemas distribuidos y manejo de concurrencia.

El uso de una arquitectura cliente-servidor facilitó la administración centralizada de los archivos y permitió que varios clientes interactuaran con el servidor de manera simultánea. Asimismo, el protocolo TCP garantizó una comunicación confiable durante la transferencia de información, evitando pérdidas de datos y asegurando la correcta recepción de los archivos enviados.

La incorporación de un mecanismo básico de versionado permitió conservar diferentes versiones de los documentos almacenados, proporcionando una funcionalidad similar a la utilizada por plataformas comerciales de almacenamiento en la nube.

Finalmente, el desarrollo del proyecto permitió comprender el funcionamiento interno de los sistemas distribuidos y sentó las bases para futuras mejoras, como la incorporación de autenticación de usuarios, cifrado de comunicaciones y sincronización automática en tiempo real.

Referencias bibliográficas (APA 7)

Canonical Ltd. (2024). *Ubuntu Server Documentation*.
<https://documentation.ubuntu.com/server/>

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.

Kurose, J. F., & Ross, K. W. (2022). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.

Stevens, W. R., Fenner, B., & Rudoff, A. M. (2004). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley.

Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). Pearson.